Visual Representation Learning with Deep Neural Networks

Tayyab Waseem
Department of Computer Science
Lahore University of Management Sciences
27100171@lums.edu.pk

https://github.com/TayyabWaseem/vision-representations-study.git

Abstract

This report acts as an investigation of the behavior of serveral deep learning architectures through a series of experiments, involving ResNet-152, Vision Transformers (ViT), CLIP, and Variational Autoencoders (VAE). The experimetns focus on different aspects of representation learning, including residual connections, transfer learning, attention, multimodal embedding and applications, latent-space, reconstruction, and generation. The results reflect how architectural choices affect model performance and representations, while also highlighting limitations of visualizations and qualitative analysis.

1 Introduction

Deep learning models can learn useful representations directly from data, but different architectures make different assumptions about how those representations should be constructed. Convolutional neural networks build hierarchical spatial features, residual networks use shortcut connections to support very deep models (a limitation of standard CNNs), and transformer-based architectures instead model relationships between input tokens. Other models, such as CLIP and VAEs, approach representation learning from different directions by connecting images with language or learning structured latent distributions.

This report examines four such architectures through a set of practical experiments. ResNet-152 is used to investigate residual connections, feature representations at different depths, and the effect of pretrained weights during transfer learning. Vision Transformer (ViT) is examined through attention visualizations, patch masking, and a comparison between the [CLS] representation and mean-pooled patch features. CLIP is evaluated through zero-shot classification and analysis of the relationship between its image and text embeddings. Lastly, we train a Variational Autoencoder on MNIST to study its latent space, reconstruction quality, generation from the prior, KL-Divergence visualization and latent interpolation (additional experiments).

The experiments are based on ideas introduced in the original ResNet (1), ViT (2), CLIP (3), and VAE (4) papers. Rather than attempting to reproduce the results of these papers, the goal is to examine the underlying behaviour of the models through smaller experiments and to compare what can be observed from their learned representations.

2 Inner Workings of ResNet-152

2.1 Methodology & Architecture

ResNet-152 is a 152-layer convolutional network built out of bottleneck residual blocks. Each bottleneck block does a 1×1 convolution to reduce the channel dimension, a 3×3 convolution at that reduced width, and a 1×1 convolution to expand it back, with batch normalization and ReLU after each conv. The part that actually makes ResNet a "residual" network is the shortcut: the input to the block is added back to the output of the third convolution before the final ReLU, so the block computes $H(x) = F(x) + x$ instead of just $F(x)$. This means the block only has to learn a residual correction on top of the identity mapping rather than reconstructing the whole transformation from scratch, which is why very deep stacks of these blocks are actually trainable; without the shortcut, gradients have to pass through dozens of stacked nonlinear layers, and in practice this leads to vanishing gradients and much slower, noisier optimization.

For all experiments the ImageNet-pretrained ResNet-152 (`IMAGENET1K_V2` weights) from torchvision was used. CIFAR-10 images (32×32) were resized to 224×224 and normalized with the standard ImageNet mean/std, since the pretrained backbone expects that input distribution. Because training on the full 50k-image CIFAR-10 training set repeatedly across several experiments would have taken a long time, all experiments used a fixed random subset of 10,000 training images, with the full 10,000-image CIFAR-10 test set used for evaluation after each epoch. Training used Adam with a learning rate of 0.001 and ran for 5 epochs in every experiment.

Four experiments were run:

1. **Baseline (head-only fine-tuning).** The final fully-connected layer was replaced with a 10-class linear layer, the rest of the backbone was frozen, and only the new head was trained.
2. **Residual ablation.** A second copy of the frozen-backbone model was created, and the `forward()` method of three bottleneck blocks (the first block of `layer2`, `layer3`, and `layer4`) was monkey-patched to remove the identity addition, so those blocks compute $F(x)$ instead of $F(x) + x$. The head was trained the same way as the baseline.
3. **Feature hierarchy visualization.** Forward hooks were attached to `layer1`, `layer3`, and `layer4` of the (unmodified, pretrained) baseline model. For roughly 1,000 test images, the hooked activations were global-average-pooled over the spatial dimensions to get a fixed-length vector per image per layer, then t-SNE was run separately on each layer's set of vectors and the 2D projections plotted, colored by class.
4. **Transfer learning comparison.** Four variants were trained: pretrained weights with only `layer4`+head unfrozen, pretrained weights with the entire backbone unfrozen, random initialization with the entire backbone trained, and random initialization with only `layer4`+head trained.

One thing worth flagging up front: the batch size was not held constant across all runs. The baseline, residual ablation, and the pretrained partial fine-tune (`layer4`+head) used a batch size of 128, but partway through the transfer-learning experiments the batch size had to be dropped to 32 to avoid an out-of-memory error, and the remaining three runs (full fine-tune, scratch, random-partial) all used batch size 32. This is a confound when comparing those runs to each other and to the earlier ones, since batch size affects the noise in the gradient estimate and effectively the number of optimizer steps per epoch.

2.2 Results

2.2.1 Baseline vs. residual ablation

The baseline loss curve drops steeply in the first two epochs and then starts to flatten out, while validation accuracy rises steadily and is starting to plateau around 82% by epoch 5. The no-skip run behaves completely differently: the loss decreases almost linearly rather than curving off, and it is still around 2.1 after 5 epochs; barely below the loss of a model making close to random 10-class predictions ($\ln 10 \approx 2.30$). Validation accuracy for the no-skip model creeps up slowly and reaches only about 30% by epoch 5, versus 82% for the baseline.

Table 1: Head-only training with intact residual connections (baseline) vs. with skip connections removed from three bottleneck blocks.

Epoch	Baseline Loss	Baseline Val Acc	No-Skip Loss	No-Skip Val Acc
1	1.2656	76.21%	2.2742	25.27%
2	0.7179	79.12%	2.2149	27.99%
3	0.5859	80.96%	2.1669	29.22%
4	0.5239	81.25%	2.1331	29.57%
5	0.4757	81.92%	2.1033	30.14%

2.2.2 Feature hierarchy across layers

Global-average-pooled activations were collected for 1,024 test images at three depths, giving feature vectors of shape (1024, 256) for the early layer, (1024, 1024) for the middle layer, and (1024, 2048) for the late layer; the increasing width simply reflects the channel counts of `layer1`, `layer3`, and `layer4`. Figure 1 shows the t-SNE projection of each set, colored by CIFAR-10 class.

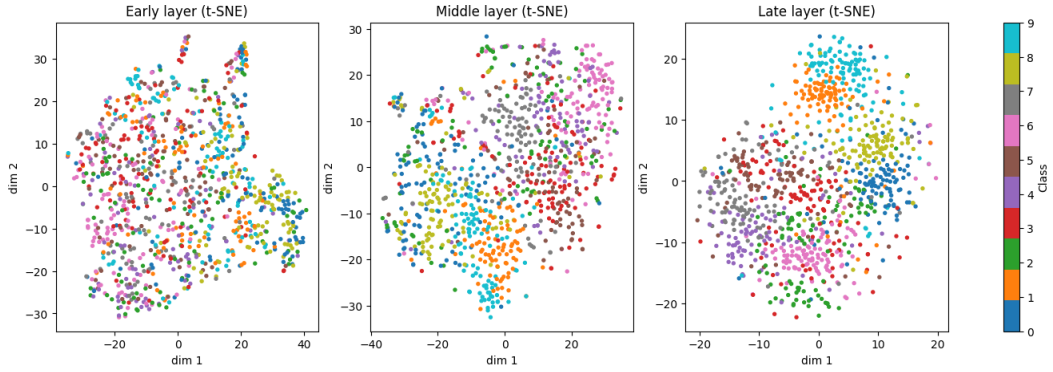

Figure 1: t-SNE projections of pooled activations from the early, middle, and late layers of the pretrained ResNet-152.

In the early-layer plot the ten classes are almost completely mixed together, with no visible grouping. The middle-layer plot shows some looser clumping; a few regions are dominated by one or two colors, but the boundaries are fuzzy and there's still a lot of mixing. In the late-layer plot several classes (for example the cyan and orange points) form clearly separated clusters, though a handful of classes such as green and pink still overlap noticeably rather than being fully separated.

2.2.3 Transfer learning comparison

Table 2: Final training loss and best validation accuracy achieved for each of the four transfer-learning settings.

Setting	Final Loss (epoch 5)	Best Val Acc
Pretrained, layer4+head (Partial FT)	0.0486	88.31% (epoch 4)
Pretrained, full backbone (Full FT)	0.2121	82.92% (epoch 5)
Random init, full backbone (Scratch)	1.4773	45.23% (epoch 4)
Random init, layer4+head (Rand-Partial)	1.8584	29.71% (epoch 5)

Figure 3 plots training loss and validation accuracy across epochs for all five configurations run in this part of the assignment (the baseline head-only run is included again alongside the four transfer settings for reference).

These are relatively surprising results, because pretrained partial fine-tune reaches the lowest loss and the highest validation accuracy of any run, and gets there fastest; by epoch 2 it is already above 86%. It does dip slightly at epoch 5 (88.31% $\rightarrow$ 86.42%), which looks like mild overfitting given how

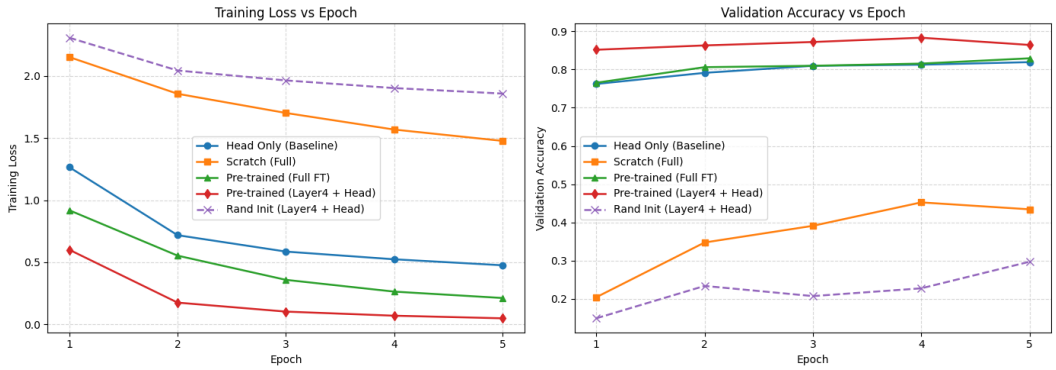

Figure 2: Training loss and validation accuracy across epochs for the baseline and the four transfer-learning configurations.

low the training loss had already gotten. Full fine-tuning of the pretrained backbone converges more slowly and ends lower (82.92%), despite updating far more parameters. Both random-initialization runs are clearly worse than either pretrained setting: full-backbone training from scratch climbs to 45.23% by epoch 4 before also dipping slightly at epoch 5, and random-init partial training barely gets past chance-level performance, ending around 20-30% and fluctuating rather than improving smoothly.

2.3 Discussion & Analysis

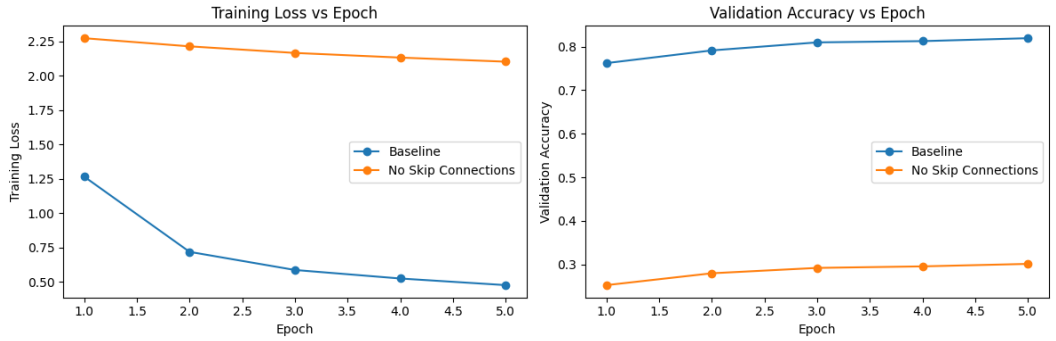

Figure 3: Training loss and validation accuracy across epochs for the baseline and model where we froze skip connections.

Training ResNet-152 from scratch on a 10k-image CIFAR-10 subset is impractical. A 152-layer network has roughly 60 million parameters, which generally requires far more data to fit well; otherwise the model fails to converge or overfits, as the "Scratch" and "Rand-Partial" results show. Pretrained ImageNet weights already encode useful visual structure, so most of the work has already been done. Freezing almost the entire backbone and training only a linear head still reaches about 82% validation accuracy in 5 epochs, showing that the pretrained features are already linearly separable enough for a simple classifier.

The residual ablation is consistent with the standard argument about gradient flow, but the experiment did not measure gradients directly. What was measured is training loss and validation accuracy, both dramatically worse without skip connections. A plausible explanation is that removing the shortcuts attenuates gradient signals reaching earlier layers. However, only the classification head was trained, so the result could also reflect changes in the statistics of features reaching the head rather than a pure gradient-flow effect. The experiment cannot distinguish between these explanations.

The t-SNE plots show increasing class separation from early to late layers, consistent with the idea that deeper layers form more semantic representations. Still, t-SNE is a nonlinear visualization

method whose distances and cluster shapes do not necessarily reflect the original feature space; it can create the appearance of structure that is not truly present. The plots are decent qualitative evidence but not a quantitative proof of separability.

In the transfer-learning comparison, pretrained weights substantially outperform random initialization regardless of how much of the backbone is unfrozen. More interestingly, partial fine-tuning (`layer4`+head) outperformed full fine-tuning in both convergence speed and final accuracy. With only 10k images and 5 epochs, unfreezing the entire backbone may give the optimizer too many parameters to adjust, risking disruption of already-useful features. The batch-size difference (128 vs. 32) is a confound here. Overall, `layer4`+head fine-tuning offers the best compute/accuracy trade-off for this dataset size.

A few limitations should be noted. The same CIFAR-10 test set served as the validation set, so the reported accuracies are not true held-out estimates. Five epochs is a short budget, and several curves had not plateaued. Similarly, like I mentioned earlier, the batch size thing was a notable limitation of using Kaggle for compute.

3 Understanding Self-attention & Vision Transformers (ViT)

3.1 Methodology & Architecture

The model used for these experiments was the pretrained `google/vit-base-patch16-224` Vision Transformer from Hugging Face. I tried doing the same experiments using a TIMM transformer `timm/vit_small_patch16_224.augreg_in1k` due to its resource efficiency and AugReg optimization, but timm models does not currently support returning attention weights so we had to abandon that idea. Unlike a CNN, which processes an image through a hierarchy of convolutional feature maps, ViT divides the image into fixed-size patches and treats them as a sequence of tokens. With an input size of 224×224 and a patch size of 16×16, each image produces $14 \times 14 = 196$ image patches. A learnable [CLS] token is added to this sequence, giving a total sequence length of 197 tokens. The transformer then processes these tokens using self-attention, allowing information from different image regions to interact before the final representation is used for classification.

The pretrained model was used directly without additional training. Three ImageNet sample images were selected: a goldfish, a box turtle, and a goose. Each image was resized to 224×224 and preprocessed before being passed to the model. The top-1 prediction was recorded, and attention outputs were enabled. The model produced 12 attention layers, each of shape $(1, 12, 197, 197)$.

For the attention visualization, the final layer's 12 heads were averaged. The attention from the [CLS] token to the 196 image patches was extracted, reshaped into a 14×14 map, upsampled to 224×224, and overlaid on the original image.

A patch-masking experiment tested robustness to information loss. Fractions of 0–90% of the 196 patches were set to black using two strategies: random masking and center masking (removing central patches first). Top-1 prediction and confidence were recorded at each level; confidence was set to zero for incorrect predictions.

The final experiment compared the [CLS] token with mean-pooled patch features. Both 768-dimensional representations were extracted from the full CIFAR-10 training set. A logistic regression classifier was trained on each using the same stratified 75/25 split.

3.2 Results

3.2.1 Image classification and attention visualization

The ViT correctly classified all three test images at the top-1 level. The goldfish image was classified as 'goldfish, Carassius auratus", the box turtle image as 'box turtle, box tortoise", and the goose image as "goose". The predictions therefore appeared reasonable for all three images rather than being obvious misclassifications.

The attention visualizations were less consistent than the classification results. In the box turtle image, the strongest regions of the heatmap were concentrated around the turtle's shell and body, with some attention extending into the surrounding grass. This is the most intuitive of the three

cases because the regions receiving the strongest attention correspond reasonably well to the object being classified. In contrast, the goldfish and goose visualizations showed much more attention on the surrounding background than on the objects themselves. The goldfish heatmap concentrated on the aquatic plants and water, while the goose heatmap placed substantial attention on the grass and the nearby edge of the sidewalk rather than the goose itself. These observations are visible in `box-turtles-heatmap.png`, `goldfish-heatmap.png`, and `goose-heatmap.png`.

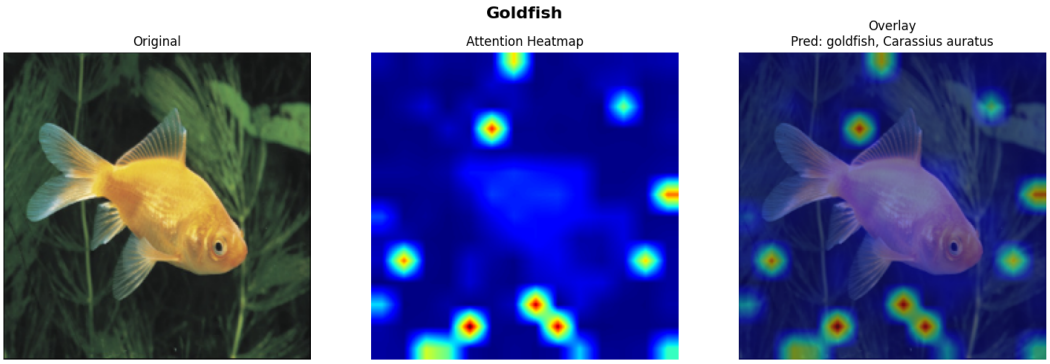

Figure 4: Original image, attention heatmap, and attention overlay for the goldfish image.

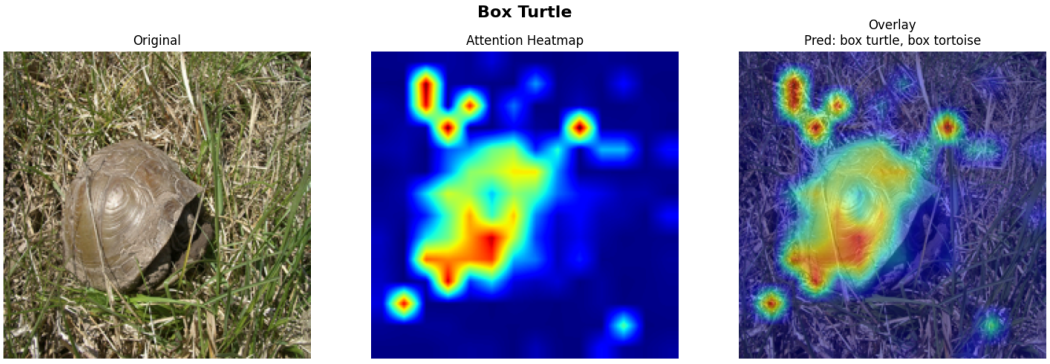

Figure 5: Original image, attention heatmap, and attention overlay for the box turtle image.

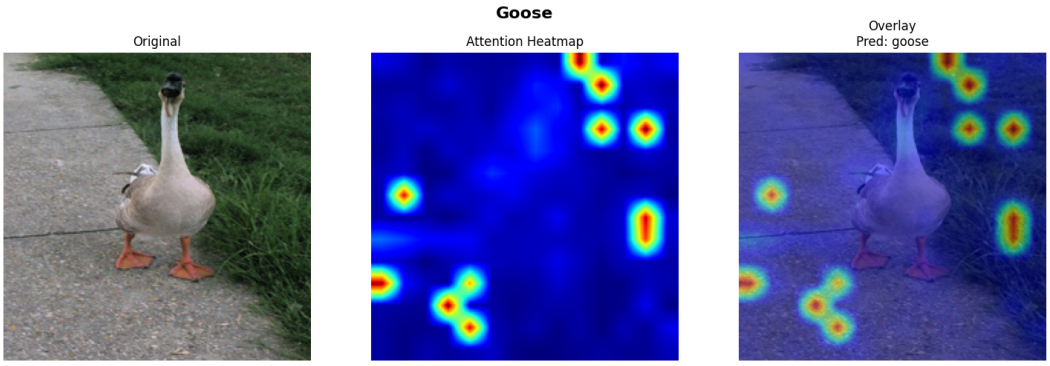

Figure 6: Original image, attention heatmap, and attention overlay for the goose image.

3.2.2 Patch masking

Masking %	Random	Center
0%	goldfish (0.998) ✓	goldfish (0.998) ✓
10%	goldfish (0.999) ✓	goldfish (0.999) ✓
25%	goldfish (0.998) ✓	goldfish (0.999) ✓
40%	goldfish (0.999) ✓	goldfish (0.987) ✓
50%	goldfish (0.999) ✓	toucan (0.045) ×
60%	goldfish (0.999) ✓	CRT screen (0.034) ×
75%	goldfish (0.999) ✓	nematode (0.085) ×
90%	goldfish (0.170) ✓	nematode (0.095) ×

Table 3: Effect of random versus center masking on ViT predictions for **Goldfish**.

Masking %	Random	Center
0%	box turtle (0.928) ✓	box turtle (0.928) ✓
10%	box turtle (0.928) ✓	box turtle (0.716) ✓
25%	box turtle (0.852) ✓	beaver (0.097) ×
40%	box turtle (0.925) ✓	armadillo (0.035) ×
50%	box turtle (0.703) ✓	echidna (0.061) ×
60%	terrapin (0.214) ×	golf ball (0.052) ×
75%	maze (0.034) ×	golf ball (0.075) ×
90%	swing (0.432) ×	armadillo (0.151) ×

Table 4: Effect of random versus center masking on ViT predictions for **Box Turtle**.

Masking %	Random	Center
0%	goose (0.948) ✓	goose (0.948) ✓
10%	goose (0.922) ✓	goose (0.093) ✓
25%	goose (0.964) ✓	pug (0.203) ×
40%	goose (0.924) ✓	skunk (0.529) ×
50%	goose (0.968) ✓	park bench (0.098) ×
60%	goose (0.808) ✓	park bench (0.070) ×
75%	mailbox (0.138) ×	park bench (0.084) ×
90%	pole (0.230) ×	mailbox (0.089) ×

Table 5: Effect of random versus center masking on ViT predictions for **Goose**.

The masking results show a clear difference between random and center masking, although the exact point at which the model fails depends strongly on the image. For the goldfish, random masking had almost no effect on the prediction until the most extreme setting. The model still predicted goldfish with confidence around 0.999 even when 75% of the patches were randomly masked, and it remained correct at 90%, although its confidence dropped to 0.170. Center masking was much more damaging: the model remained correct through 40% masking, but at 50% it changed to "toucan" and remained incorrect at all higher masking levels.

The box turtle showed a similar pattern but with a lower tolerance for random masking. Random masking preserved the correct prediction through 50% masking, although confidence fell to 0.703 at that point. At 60%, the prediction changed to 'terrapin", followed by 'maze, labyrinth" at 75% and 'swing" at 90%. Center masking caused an earlier failure: the model was still correct at 10%, but already predicted 'beaver" at 25% and remained incorrect for every higher masking level.

The goose produced the strongest random-masking robustness of the three images. The model remained correct with confidence above 0.80 through 60% random masking. It failed at 75%, predicting 'mailbox", and then predicted 'pole" at 90%. Center masking again caused a much faster degradation. The model was technically still correct at 10% masking, but its confidence had already fallen from 0.948 to 0.093. At 25%, the prediction changed to "pug", and every subsequent center-masked image was classified incorrectly.

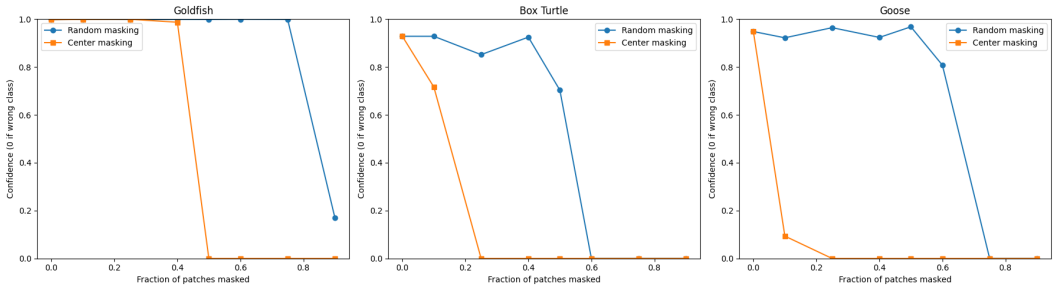

Figure 7: Effect of random and center patch masking on confidence for the goldfish, box turtle, and goose images. Confidence is shown as zero when the predicted class is incorrect.

3.2.3 CLS token vs. mean-pooled patch tokens

The linear-probe experiment produced very similar results for the two representations. The [CLS] token achieved an accuracy of 0.9597 on the 12,500-image test split, while mean-pooling the 196 patch tokens achieved 0.9633. The difference was therefore only -0.0036 when calculated as CLS $-$ Mean, meaning the mean-pooled representation was slightly better in this experiment.

Table 6: Linear-probe comparison between the final [CLS] representation and the mean of the final patch tokens on CIFAR-10.

Representation	**Linear Probe Accuracy**
[CLS] token	95.97%
Mean-pooled patch tokens	96.33%
Difference (CLS - Mean)	-0.36 percentage points

The per-class results were also close. For the [CLS] representation, the lowest F1-score was 0.90 for class 3, while the mean-pooled representation achieved 0.92 for the same class. Most of the remaining classes had F1-scores between 0.93 and 0.98 for both representations. This makes the overall accuracy difference fairly small rather than evidence of a large advantage for either pooling method.

3.3 Discussion & Analysis

The classification results show that the pretrained ViT handled the three selected ImageNet examples correctly, but the attention maps make the result more complicated than simply saying that the model correctly "looked at" each object. The box turtle provides the expected case: its strongest attention regions overlap the turtle itself. The goldfish and goose are different. In both cases the model predicted the correct class while the averaged final-layer [CLS] attention concentrated largely on the background. Correct classification therefore does not necessarily require the attention visualization to align with the human-defined object.

A plausible explanation is that the model has learned contextual correlations useful for ImageNet classification. A goldfish is often associated with water and aquatic plants, while a goose may commonly appear around grass. If these contextual features are sufficiently predictive, the model can use them even when they are not part of the object's intrinsic appearance. This remains an interpretation: the experiment did not test the model on goldfish or geese in different backgrounds, so it does not demonstrate failure under a background shift.

The attention visualization is useful because the ViT already computes attention relationships as part of its forward pass. The [CLS]-to-patch attention can be extracted without a separate gradient-based procedure. A CNN does not provide an equivalent matrix by default, so methods such as Grad-CAM are typically used instead. The ViT heatmaps are convenient for inspection, but they should not automatically be treated as causal explanations of the prediction. The goldfish and goose examples illustrate the distinction: the model was correct even though the highlighted regions did not correspond closely to the objects.

The masking results reinforce the importance of image structure. Random masking was consistently less damaging than center masking. The difference was especially large for the goose: the model remained correct through 60% random masking, whereas center masking reduced confidence to 0.093 at 10% and produced an error at 25%. The goldfish and box turtle showed the same pattern, with center masking causing errors earlier than random masking. The spatial location of the removed information therefore matters substantially.

There is a caveat with the center-masking design. It assumes the center is important because ImageNet images are often center-biased, but the experiment does not independently establish that the central patches are the ones the model relies on most. Center masking is better described as a structured occlusion test than as a direct probe of the model's most important regions.

The [CLS] versus mean-pooling experiment produced a much smaller difference than might be expected. Mean-pooled patch features performed slightly better (96.33% vs. 95.97%). The 0.36-point gap is small, so the result does not provide strong evidence that mean pooling is generally superior; it only shows that, for this pretrained ViT and this CIFAR-10 linear-probe setup, averaging the patch representations retained enough class information to match or exceed the dedicated classification token.

Overall, the experiments show three properties of the ViT. The model was highly accurate on the three ImageNet examples, yet its attention maps did not always align with the objects. It remained surprisingly robust to random masking while failing much earlier under center masking. Both the [CLS] and mean-pooled representations transferred well to CIFAR-10, with only a small difference between them. These results are useful demonstrations, but the small number of selected images and the qualitative nature of the attention analysis limit stronger claims about general model behavior.

4 Contrastive Learning & CLIP

4.1 Methodology & Architecture

CLIP (Contrastive Language-Image Pre-training) is a multimodal model trained with a contrastive objective that pulls matching image-text pairs closer together in a shared embedding space while pushing non-matching pairs apart.

The main experiment was zero-shot classification on the STL-10 test set (ten classes: airplane, bird, car, cat, deer, dog, horse, monkey, ship, truck). Instead of training a classifier, CLIP encoded both images and candidate class descriptions into its joint embedding space. The predicted class was simply the text representation with the highest cosine similarity to the image embedding. Three prompting strategies were compared: plain class names, the template "a photo of a [class]", and a longer descriptive template. Both image and text embeddings were normalized before computing similarities.

```
prompts = {
    "i. Plain labels": [f"{c}" for c in classes],
    "ii. Prompted text": [f"a photo of a {c}" for c in classes],
    "iii. Descriptive variants": [f"a centered, high quality photo of a {c},
        showcasing its distinct features" for c in classes]
}
```

The first experiment used the complete STL-10 test set. Each image embedding was compared with the ten candidate text embeddings, converted to a softmax distribution, and assigned the class with the highest value.

The second experiment examined the modality gap that can appear even after contrastive training. A random batch of 100 STL-10 images was paired with the corresponding "a photo of a [class]" prompts. Image and text embeddings were extracted, normalized, and stacked before t-SNE so that both modalities appeared in the same projection. Dashed lines linked corresponding image-text pairs.

The final experiment attempted to bridge this residual gap with an orthogonal Procrustes transformation. A training batch was used to learn a matrix R minimizing $\|XR - Y\|_F$, where X and Y are image and text embeddings. The matrix was applied to a separate set of test image embeddings, which

were then visualized with their text counterparts via t-SNE. Zero-shot classification was repeated after the transformation to measure any change in accuracy.

4.2 Results

4.2.1 Zero-shot classification

The zero-shot results were already very strong before any attempt to align the two modalities. The plain-label prompts achieved 96.26% accuracy on STL-10, while the "a photo of a [class]" prompts performed best at 97.36%. The more descriptive prompts returned to 96.26%, so adding substantially more wording did not improve the result.

Table 7: Zero-shot classification accuracy on the STL-10 test set for the three prompting strategies.

Prompting strategy	**STL-10 Accuracy**
Plain labels	96.26%
Prompted text	97.36%
Descriptive variants	96.26%

The relatively small difference between the three settings is interesting because the prompts differ quite a lot in wording. The simple class names were already sufficient to reach above 96%, while the standard "a photo of a [class]" template provided a modest improvement. The longer descriptive template did not provide any additional benefit and in fact returned to the same accuracy as the plain labels.

4.2.2 Modality gap

The unaligned embedding visualization showed a pronounced separation between the two modalities. The image embeddings formed a compact region on one side of the t-SNE projection, while the text embeddings occupied a separate region with a visible gap between them. The corresponding image-text pairs were connected with dashed lines, and many image points connected to the same text location because images from the same class were all mapped to the identical text prompt. For example, every image labelled as a dog was paired with the same text string "a photo of a dog", so the text side contains one representation for that class while the image side contains many different visual examples.

The plot therefore gives qualitative evidence of a modality gap, but the size of the gap should not be interpreted directly from the distances in the two-dimensional figure. t-SNE is nonlinear and is designed primarily to preserve local neighborhoods, so the apparent physical distance between the image and text clusters does not correspond directly to their cosine distance in CLIP's original embedding space. What can be said more safely is that the two modalities occupy clearly different regions in this projection while still maintaining their own internal structure.

4.2.3 Bridging the modality gap

After learning the Procrustes transformation, the aligned t-SNE visualization looked substantially different. The image and text embeddings became heavily intermixed, with corresponding representations appearing in much closer local neighborhoods than before. The transformation therefore changed the relative placement of the two modalities without requiring the CLIP encoders themselves to be retrained.

Figure 8: t-SNE projection of the original CLIP image and text embeddings, showing the separation between the two modalities.

Figure 9: Distribution of cosine similarities for matching image-text pairs before and after Procrustes alignment.

The more quantitative results point in the same direction. Before alignment, the histogram of cosine similarities for matching image-text pairs was concentrated roughly between 0.2 and 0.3. After applying the Procrustes transformation, the distribution shifted substantially, with matching-pair similarities reported in the approximate range of 0.75 to 0.95. This is a much larger change than the small improvement in zero-shot accuracy might suggest.

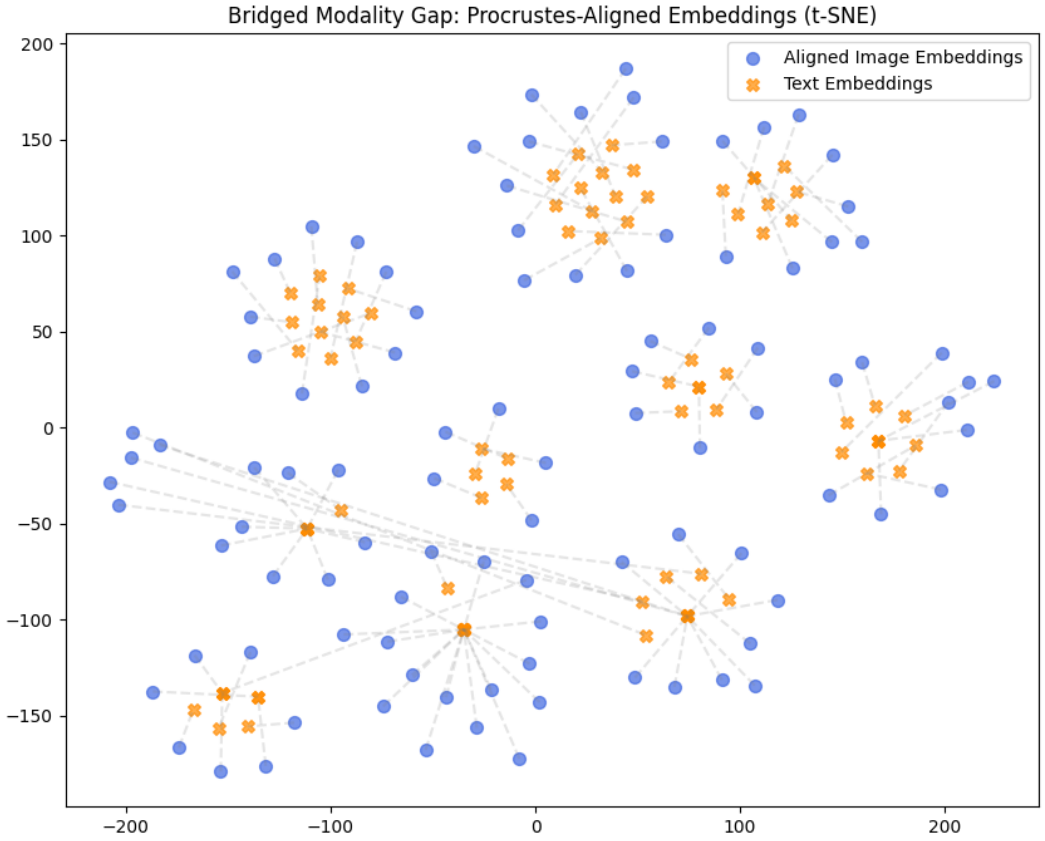

Figure 10: t-SNE projection after Procrustes alignment of the CLIP image embeddings.

The zero-shot accuracy also increased after alignment. The plain-label strategy improved from 96.26% to 96.70%, the standard prompted version increased from 97.36% to 97.74%, and the descriptive prompt improved more noticeably from 96.26% to 97.59%.

Table 8: Zero-shot accuracy before and after applying the learned Procrustes transformation to image embeddings.

Prompting strategy	**Original**	**Procrustes-aligned**
Plain labels	96.26%	96.70%
Prompted text	97.36%	97.74%
Descriptive variants	96.26%	97.59%

The similarity matrix provides another view of the classification result. It showed a strong diagonal pattern when image embeddings were compared with the ten text prompts, meaning that images generally received their highest similarity from the correct class prompt. The individual examples shown in `Figure 11` follow the same pattern, with the correct class receiving nearly all of the softmax probability for the displayed airplane, bird, and car examples.

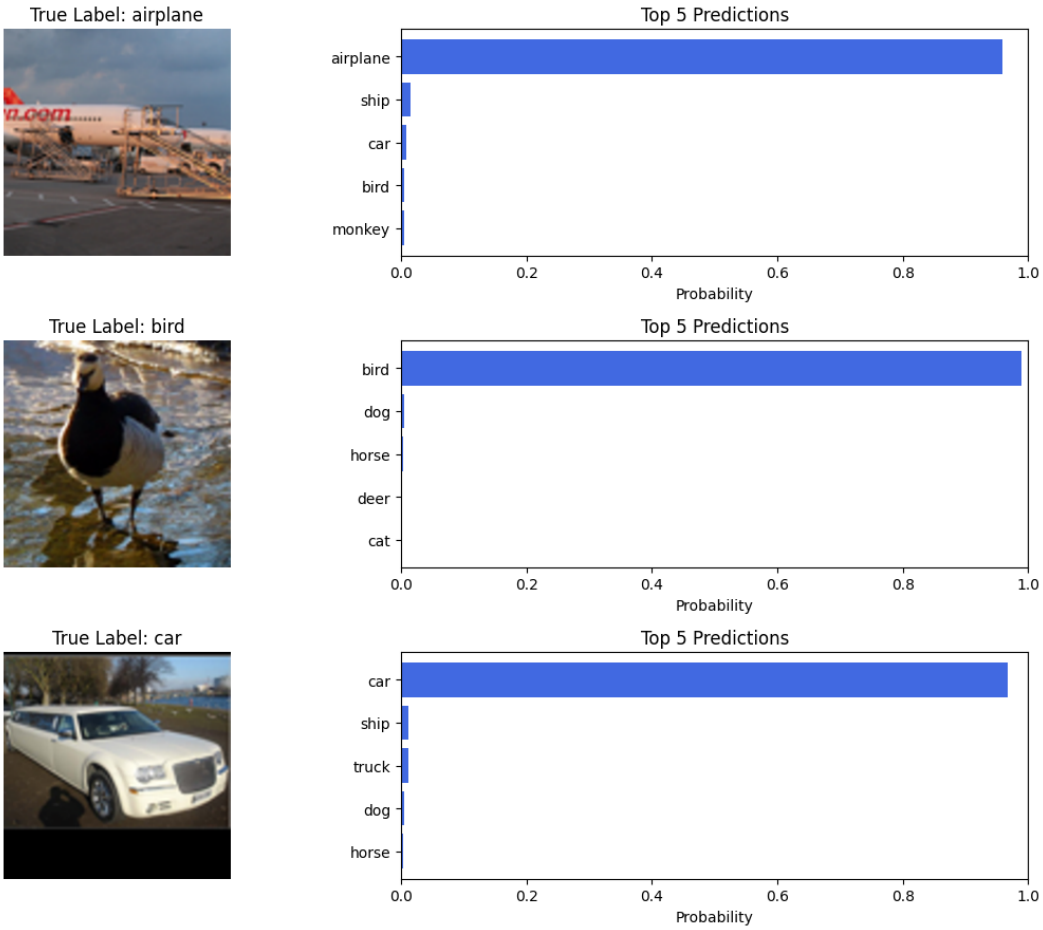

Figure 11: Example zero-shot predictions showing the model's probabilities for individual STL-10 images.

4.3 Discussion & Analysis

The most straightforward result is that CLIP performed extremely well on STL-10 without any task-specific training. The best prompt ("a photo of a [class]") reached 97.36%, and even plain class names achieved 96.26%. More descriptive language was not automatically better.

The modality-gap visualization is more surprising. Although CLIP is trained to make matching image-text pairs similar, the t-SNE projection places the two modalities in separate regions. This is compatible with high zero-shot accuracy: classification only requires the correct text to rank above the alternatives, not that every image embedding occupy the same region as its text counterpart.

Even when absolute matching similarities were relatively low, the correct class generally remained the most similar. Relative ranking, rather than large absolute values, was sufficient for the observed 96–97% accuracy.

The Procrustes experiment applied a simple orthogonal transformation ($X_{\text{aligned}} = XR$) with no explicit translation. It produced a large rise in pairwise similarity (approximately 0.25 to 0.85) but only a modest change in classification accuracy. The original space was already well organized for the task; closing the geometric gap mainly made the representations more similar while preserving existing performance.

Overall, CLIP does not need image and text embeddings to occupy the same coordinate region for effective zero-shot classification. The absolute modality gap appears less important than the relative structure the model preserves. These conclusions are limited to the tested embeddings and a single alignment procedure.

5 Variational Autoencoders (VAE)

5.1 Methodology & Architecture

The model used in this experiment was a Variational Autoencoder (VAE) trained on the MNIST dataset. The VAE consists of an encoder, a latent representation, and a decoder. Unlike a standard autoencoder, the encoder does not produce a single deterministic latent vector. Instead, it produces the mean μ and log-variance $\log \sigma^2$ of a Gaussian distribution $q_\phi(z|x)$. A latent vector is then sampled using the reparameterization trick, $z = \mu + \sigma \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$. This allows the sampling operation to remain differentiable during training.

The encoder first flattens each 28×28 MNIST image into a 784-dimensional vector and passes it through a fully connected layer with 400 neurons and a ReLU activation. Two separate linear layers then produce the 20-dimensional μ and $\log \sigma^2$ vectors. The decoder takes the sampled 20-dimensional latent vector, maps it through another 400-neuron ReLU layer, and produces 784 output values. A sigmoid activation converts these values into pixel probabilities in the range $[0, 1]$. The model was trained for 15 epochs using Adam with a learning rate of 10^{-3} and a batch size of 128.

The training objective was the negative ELBO, consisting of a binary cross-entropy reconstruction term and a KL-divergence term. The reconstruction loss measures how closely the decoder output matches the original image, while the KL term encourages the learned latent distributions to remain close to the standard normal prior $\mathcal{N}(0, I)$. The loss used in the implementation was

$$\mathcal{L} = \mathrm{BCE}(x, \hat{x}) + D_{\mathrm{KL}}\big(q_\phi(z|x) \| \mathcal{N}(0, I)\big).$$

After training, three main analyses were performed. First, the 20-dimensional latent means μ of 2,000 randomly selected MNIST test images were projected into two dimensions using t-SNE and visualized according to their digit labels. Second, ten test images were passed through the complete VAE and their reconstructions were compared with the original images. Finally, ten random vectors were sampled directly from the prior $\mathcal{N}(0, I)$ and passed through the decoder to generate new handwritten digits.

Two additional analyses were also considered. The first examined the distribution of the learned latent means by comparing them with the standard normal distribution. The second performed a latent-space interpolation between two points, producing a sequence of decoded images between the corresponding latent representations.

5.2 Results

5.2.1 Training behaviour

The negative ELBO decreased consistently throughout training. It started at 165.7979 in epoch 1 and dropped sharply to 121.9434 by epoch 2. After this initial decrease, the improvement became progressively smaller, with the loss reaching 110.0134 at epoch 5 and 104.8741 at epoch 15.

The largest improvement occurred during the first few epochs. From epoch 1 to epoch 2, the loss decreased by almost 44 points, whereas the decrease from epoch 14 to epoch 15 was only about 0.25. This indicates that the model learned the basic structure of the MNIST images relatively quickly and then made smaller improvements as training continued. Since only the training loss was recorded, this experiment does not establish whether the model was still improving on unseen data at the end of training.

5.2.2 Latent space

The t-SNE projection of the latent means showed visible grouping according to digit class. Several digits formed fairly distinct clusters, such as the clusters corresponding to 0 and 6, although many of the class regions touched or overlapped. The result therefore suggests that the VAE's latent representation contains information about digit identity without producing completely separated regions for every class.

The overlap between classes is not necessarily a failure of the representation. MNIST contains visually similar digits, and the VAE is not trained with an explicit classification objective. Its objective is to reconstruct the input while keeping the latent distribution close to the prior. As a result, there is no

Table 9: Average negative ELBO during VAE training.

Epoch	Average Negative ELBO
1	165.7979
2	121.9434
3	114.7772
4	111.7920
5	110.0134
6	108.8384
7	107.9663
8	107.2955
9	106.7229
10	106.3206
11	105.9672
12	105.6369
13	105.3471
14	105.1276
15	104.8741

direct reason for all examples belonging to the same digit to occupy completely separate regions. The t-SNE plot only provides a qualitative visualization of the representation, so the apparent cluster separation should not be interpreted as a quantitative measurement of latent-space separability.

The additional latent-distribution experiment showed that the learned latent means were distributed approximately around zero and followed a shape similar to the target standard normal distribution. The histogram was broadly centered around zero and followed the general bell-shaped form of the $\mathcal{N}(0, 1)$ reference curve.

This is consistent with the role of the KL term in the VAE objective. However, the histogram alone does not measure the KL divergence of the full 20-dimensional posterior distributions. It only provides a visual comparison of the latent means against a one-dimensional standard normal distribution. The apparent agreement therefore supports the intended regularization behaviour, but it should not be treated as a direct measurement of the total KL term.

5.2.3 Reconstruction quality

The reconstruction experiment showed that the VAE was able to preserve the overall identity and structure of the input digits. The ten original test images had labels 5, 8, 0, 2, 6, 4, 6, 2, 9, and 8. Their corresponding reconstructions generally retained the main strokes and shapes of the original digits, making the reconstructed digits recognizable.

The main difference between the two rows is that the reconstructions are noticeably softer and blurrier than the originals. Fine details and sharper edges are lost, although the larger structure of each digit is generally preserved. This is expected from the reconstruction objective because the decoder is learning to represent a distribution over images through a relatively low-dimensional latent space rather than simply copying the input pixels directly.

The examples do not provide enough evidence to determine whether particular digit classes are consistently reconstructed better or worse than others. Some individual reconstructions appear cleaner than others, but this is a qualitative observation from only ten images rather than a class-level evaluation.

5.2.4 Generation from the prior

The decoder was also used without an input image by sampling ten random latent vectors from $\mathcal{N}(0, I)$. The resulting images were mostly recognizable as handwritten digits. The generated samples included recognizable forms resembling digits such as 0, 1, 2, 3, 4, 7, and 8, although the images were somewhat blurry.

This result is different from simply reconstructing an existing MNIST image. In the reconstruction experiment, the encoder provides a latent representation derived from a real image. Here, the decoder

Figure 12: t-SNE projection of the 20-dimensional latent means, with points labelled according to their MNIST digit class.

receives vectors sampled directly from the prior, so the fact that the outputs are recognizable digits indicates that the decoder has learned a meaningful relationship between regions of the latent space and plausible MNIST-like images. At the same time, the visible blurriness and variation in quality show that the generated samples are not identical in quality to the original training examples.

5.2.5 Latent Interpolation (Additional experiment)

The interpolation experiment produced a smooth sequence between a digit 2 and a digit 7. The first image was a clear 2, while the final image was a clear 7. The intermediate samples gradually changed the shape of the digit, with the lower loop of the 2 becoming progressively flatter until it developed into the more vertical structure of the 7.

This is useful evidence that the decoder has learned a relatively smooth mapping from latent variables to images. The intermediate images are not simply arbitrary combinations of the two endpoints; instead, the visual structure changes gradually along the interpolation. However, this is a single interpolation example, so it is not enough to conclude that the entire latent space has this property.

5.3 Discussion & Analysis

The training loss provides the clearest evidence that the VAE learned a useful representation of MNIST. The negative ELBO decreased over 15 epochs, with the largest drop between the first two epochs and slower gains thereafter. Without validation or test reconstruction loss, it is impossible to tell whether later epochs improved generalization or merely continued fitting the training objective.

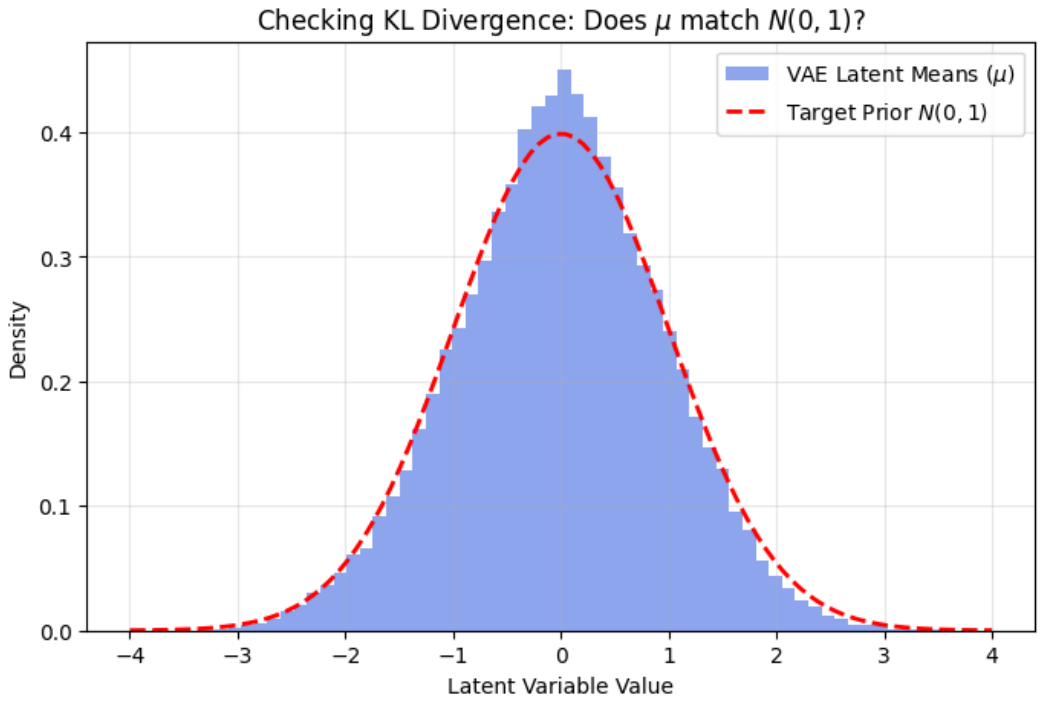

Figure 13: Distribution of the VAE latent means compared with the standard normal distribution $\mathcal{N}(0, 1)$.

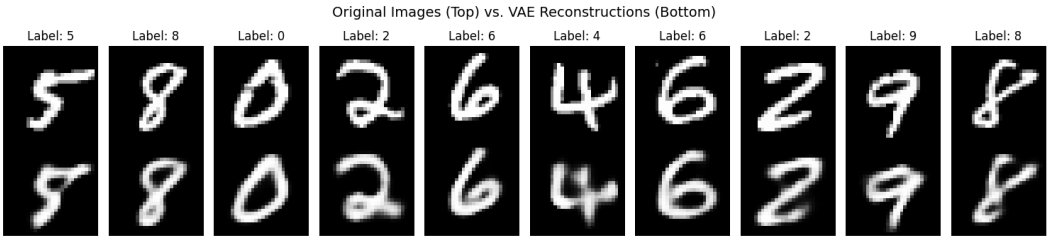

Figure 14: Original MNIST test images (top row) and their corresponding VAE reconstructions (bottom row).

The latent-space visualization offers some indication of what the encoder learned. Digit classes occupy recognizable regions in the t-SNE projection, though the clusters are not fully separated. This is expected: the VAE was never trained to classify digits or maximize class separation, only to reconstruct images while regularizing the latent distribution toward a standard normal. The emergence of class structure therefore suggests that digit-specific information arose naturally.

Reconstruction results show a similar trade-off. The model retains the major structure of the digits but loses sharpness, as seen in `original-vs-reconstruction.png`. It captures enough information to recover digit identity, yet the outputs are not pixel-for-pixel copies. A low reconstruction error does not imply that every visual detail has been preserved.

Generation is more demanding because the decoder must synthesize an image from a random prior sample. Most of the ten generated digits are recognizable, indicating that the decoder maps prior samples to a meaningful region of image space. Their blurriness matches the reconstruction results, suggesting the same underlying limitation rather than a separate failure.

The additional experiment i conducted: Latent interpolation, further supports a continuous representation: the smooth transition from 2 to 7 shows that nearby latent points can produce gradual

Figure 15: Digits generated by sampling latent vectors from the standard normal prior and passing them through the trained decoder.

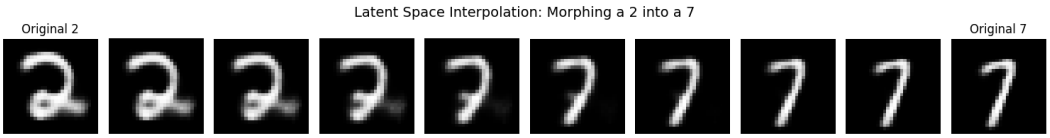

Figure 16: Latent-space interpolation between a digit 2 and a digit 7.

structural change. This is a key distinction from simple memorization. However, only a single pair was examined, so the behaviour cannot be generalized.

The KL-divergence analysis must also be read cautiously. The latent means appear centered near zero and roughly normal, consistent with the KL term, yet the experiment neither plots the KL contribution nor computes the full divergence over all 20 dimensions. Matching the distribution of μ alone does not guarantee that $q_\phi(z|x)$ matches the prior; the learned variances also matter.

Overall, the results are consistent with a structured and usable latent representation of MNIST. Decreasing ELBO, recognizable reconstructions, plausible prior samples, class-related structure, and smooth interpolation all support this view. At the same time, the evidence beyond the training loss is largely qualitative. Without quantitative reconstruction metrics, class-wise generation scores, held-out ELBO, or a full latent KL measurement, conclusions should remain limited to the observed behaviour rather than a comprehensive evaluation of the model.

References

[1] He, K., Zhang, X., Ren, S. & Sun, J. (2016) Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

[2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. & Houlsby, N. (2021) An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*.

[3] Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. & Sutskever, I. (2021) Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pp. 8748–8763.

[4] Kingma, D.P. & Welling, M. (2014) Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*.

[5] Chen, T., Kornblith, S., Norouzi, M. & Hinton, G. (2020) A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1597–1607.

[6] OpenAI (2021) CLIP: Connecting text and images. *OpenAI Blog*.